E-Invoice Platform

Integration Assessment & Scoping Guide

Document Type: Internal Use Only
For: Sales & Technical Team
Version: 1.0
Date: May 20, 2026

Purpose

This guide helps you assess integration complexity, estimate effort, and provide accurate quotes to clients. Use this **after receiving** the client's pre-onboarding questionnaire.

Quick Assessment Framework

Step 1: Identify System Type

Match the client's system to one of these categories:

System Category	Examples	Typical Integration Path
Modern POS with API	StoreHub, Square, Lightspeed	REST API integration
Malaysian ERP	SQL Accounting, AutoCount, Million	Database connector or CSV export
E-commerce Platform	WooCommerce, Shopify, Magento	Webhook + plugin
Enterprise ERP	SAP, Oracle, Microsoft Dynamics	Middleware/API integration
Custom .NET System	Bespoke applications	Code-level integration
Legacy/Old System	DOS-based, FoxPro, dBase	File export + batch import
Manual Process	Excel invoicing	CSV template + manual upload

Integration Complexity Matrix

Low Complexity (2–4 hours)

Characteristics:

- System has documented REST API
- Pre-built adapter available or easily customizable
- Standard data format (JSON/CSV)
- Client has technical staff who can configure API calls

Examples:

- WooCommerce with official plugin
- Shopify webhook integration
- Modern POS with OAuth2 API
- System that can export clean CSV files

Integration Approach:

- Install pre-built adapter (if exists)
- OR: Provide API documentation, client implements
- OR: Simple CSV template + scheduled export

Quote Range: RM 800 – RM 1,500 (if any custom work needed)

Medium Complexity (4–12 hours)

Characteristics:

- Database accessible but no API
- File export available but requires transformation
- Custom field mapping needed
- Scheduled batch processing required
- Webhook implementation needed

Examples:

- SQL Accounting (database polling)
- AutoCount (CSV export + field mapping)
- Custom POS with SQL Server backend
- Legacy system with daily data export

Integration Approach:

- Database connector (direct SQL queries or stored procedures)

- File watcher + CSV/XML parser with custom mapping
- Scheduled job (hourly/daily batch import)
- API key authentication + validation logic

Development Tasks:

1. Database schema analysis (1 hour)
2. Data extraction logic (2-4 hours)
3. Field mapping + validation (2-3 hours)
4. Scheduling + error handling (1-2 hours)
5. Testing + documentation (2-3 hours)

Quote Range: RM 1,500 – RM 4,500

High Complexity (12–40 hours)

Characteristics:

- Enterprise system with complex data model
- Multiple systems must be integrated
- Real-time synchronization required
- Custom business logic needed
- System behind firewall (VPN/tunnel required)
- Heavy data transformation

Examples:

- SAP/Oracle ERP integration
- Multi-location retail chain (5+ POS systems)
- Legacy mainframe system
- Custom .NET system requiring code modifications
- Multi-system orchestration (POS + inventory + CRM)

Integration Approach:

- Enterprise service bus (ESB) or middleware layer
- Real-time event-driven architecture
- Complex ETL (Extract, Transform, Load) pipeline
- VPN tunnel setup (if firewall restricted)
- Custom business rules engine

Development Tasks:

1. Requirements gathering + analysis (4-8 hours)

2. Architecture design (2-4 hours)
3. Integration layer development (8-16 hours)
4. Data transformation logic (4-8 hours)
5. Error handling + retry logic (2-4 hours)
6. Testing (4-8 hours)
7. Documentation + handover (2-4 hours)

Quote Range: RM 4,500 - RM 15,000

Integration Assessment Checklist

Use this during scoping call with client:

Question 1: Data Export Capability

Q: Can your system export invoice data automatically?

- ☑️ **Yes, via API** → Likely low complexity
- ☑️ **Yes, via scheduled CSV/XML export** → Medium complexity
- ☑️ **Yes, but requires manual export** → Medium (automation needed)
- ☑️ **No, data trapped in system** → High complexity

Question 2: System Access

Q: Do we have access to the system's database or API?

- ☑️ **API with documentation** → Low-medium complexity
- ☑️ **Database accessible** → Medium complexity
- ☑️ **File system only** → Medium-high complexity
- ☑️ **No access (vendor-locked)** → Contact vendor or high complexity workaround

Question 3: Data Quality

Q: Does the system capture all LHDN-required fields?

Review client's sample invoices for:

- [ ] Seller TIN, name, address, MSIC
- [ ] Buyer TIN (for transactions > RM 10,000)
- [ ] Buyer name and address
- [ ] Invoice number, date, currency

- ☐ Line items with quantities, unit prices
- ☐ Tax type and tax rate per item
- ☐ Classification codes (product/service codes)

If 1-2 fields missing: Medium complexity (auto-fill from product master)
If 3+ fields missing: High complexity (client must fix data source first)

Question 4: Volume & Frequency

Q: How many invoices and how often?

- **< 50/month:** Manual upload acceptable (no custom work)
- **50-500/month:** Batch integration (scheduled 1-2x daily)
- **500-2,000/month:** Batch integration with high frequency (hourly)
- **> 2,000/month:** Real-time integration required

Question 5: Technical Resources

Q: Does client have in-house developer/IT staff?

- ☑️ **Yes, with API experience** → They can implement, we provide docs (no charge)
- ☑️ **Yes, but no API experience** → We implement, they maintain (charge for initial)
- ☑️ **No technical staff** → Full-service integration (charge for dev + maintenance)

Integration Recommendation Template

After assessment, document your findings:

```
INTEGRATION ASSESSMENT REPORT

Client: [Name]
Date: [Date]
Assessed by: [Your name]

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

CURRENT SYSTEM
  System: [Name & version]
  Type: [POS / ERP / E-commerce / Custom]
  Hosting: [Cloud / On-premise / Hybrid]
  Data Access: [API / Database / File export / None]
```

DATA QUALITY ASSESSMENT
✅ Fields present: [List complete fields]
⚠️ Fields missing: [List missing fields]
📊 Sample quality: [Good / Fair / Poor]

INTEGRATION COMPLEXITY: [LOW / MEDIUM / HIGH]

RECOMMENDED APPROACH
Method: [API integration / Database connector / File batch / Manual]
Frequency: [Real-time / Hourly / Daily / Manual]
Data Flow: [System → E-Invoice Platform → LHDN]

ESTIMATED EFFORT
Development: [X-Y] hours
Testing: [X] hours
Documentation: [X] hours
Total: [X-Y] hours

INVESTMENT ESTIMATE
Fixed price: RM [Amount]
OR Hourly: RM [Amount] (@ RM 400/hour)

Includes:
- Initial development
- Testing & validation
- Deployment
- 1-week post-deployment support
- Basic documentation

TIMELINE
Kickoff: [Date]
Development complete: [Date]
Testing complete: [Date]
Go-live: [Date]

DEPENDENCIES
From client:
- [ ] Database credentials / API keys
- [ ] Test environment access
- [ ] Sample data (10+ invoices)

```
  - [ ] Technical contact availability

  From SteadyDevs:
  - [ ] SOW signed
  - [ ] 50% deposit received
  - [ ] Developer assigned

RISKS & ASSUMPTIONS
  - Client system documentation is accurate
  - Test environment mirrors production
  - Client provides timely feedback during development
  - [Add any specific risks]

NEXT STEPS
  1. [ ] Share this assessment with client
  2. [ ] Client approves approach & budget
  3. [ ] Generate SOW for signature
  4. [ ] Schedule kickoff meeting
```

Common Integration Patterns

Pattern 1: Database Polling (Medium Complexity)

Use Case: SQL Accounting, AutoCount, custom systems with SQL Server/MySQL

Architecture:

```
┌─────────────────┐
│ Client Database │  (invoices table)
└────────┬────────┘
         │ SQL Query every 30 min
         ↓
┌─────────────────┐
│ Polling Service │  (our custom adapter)
└────────┬────────┘
         │ POST /api/v1/adapter/import
         ↓
┌─────────────────┐
```

```
│ E-Invoice API   │
└─────────────────┘
```

Development Checklist:

- [ ] Obtain database schema
- [ ] Write SELECT query to fetch new/updated invoices
- [ ] Map database columns to API fields
- [ ] Implement polling service (Hangfire recurring job or separate console app)
- [ ] Error handling: log failed records, retry logic
- [ ] Deploy as Windows Service or Docker container

Estimated Hours: 6-10 hours

Pattern 2: File Watcher (Medium Complexity)

Use Case: Legacy systems that export daily CSV/XML files

Architecture:

```
┌─────────────────┐
│ Client System   │ exports daily CSV to shared folder
└────────┬────────┘
         │ File export (daily at 11 PM)
         ↓
┌─────────────────┐
│ Shared Folder   │ (Google Drive, Dropbox, SFTP)
└────────┬────────┘
         │ Monitored by file watcher
         ↓
┌─────────────────┐
│ Import Service  │ (our custom parser)
└────────┬────────┘
         │ POST /api/v1/adapter/import
         ↓
┌─────────────────┐
│ E-Invoice API   │
└─────────────────┘
```

Development Checklist:

- [ ] Define CSV/XML format with client
- [ ] Build parser (handle encoding, delimiters, date formats)

- [ ] Implement file watcher (FileSystemWatcher or cron job)
- [ ] Archive processed files
- [ ] Email notification on parse errors
- [ ] Deploy to client's server or our infrastructure

Estimated Hours: 5-8 hours

Pattern 3: REST API Push (Low Complexity)

Use Case: Modern POS/ERP with API capabilities, client has dev team

Architecture:

```
┌─────────────────┐
│ Client System   │ (POS / E-commerce)
└────────┬────────┘
         │ On invoice created event
         │ Client's code makes API call
         ↓
┌─────────────────┐
│ E-Invoice API   │ POST /api/v1/adapter/import
│ (X-API-Key auth)│
└─────────────────┘
```

Development Checklist:

- [ ] Generate API key for client
- [ ] Provide API documentation + code samples
- [ ] Client implements HTTP POST in their system
- [ ] Test with sample payloads
- [ ] Monitor first 100 submissions for errors

Estimated Hours: 2-4 hours (mostly support, client does the work)

Pattern 4: Webhook + Plugin (Low Complexity)

Use Case: WooCommerce, Shopify, Magento

Architecture:

```
┌─────────────────┐
│ E-commerce      │ Order completed event
│ (WooCommerce)   │
```

```
└──────────────┬──────────────┘
               │ Webhook fires
               ↓
┌─────────────────────────────┐
│ Our Plugin                  │ (PHP / JavaScript)
└──────────────┬──────────────┘
               │ POST /api/v1/adapter/import
               ↓
┌─────────────────────────────┐
│ E-Invoice API               │
└─────────────────────────────┘
```

Development Checklist:

- [ ] Install WooCommerce plugin (if pre-built)
- [ ] OR: Develop plugin using platform SDK
- [ ] Configure webhook endpoint in e-commerce admin
- [ ] Map order fields to invoice fields
- [ ] Test with dummy orders

Estimated Hours: 2-6 hours (depending on plugin availability)

Red Flags — When to Say No

🚩 **Refuse or charge premium if:**

1. **Client expects full-service integration but won't pay**
    - "Can't you just make it work for free?"
    - **Response:** Integration = custom software development = billable
2. **System is completely undocumented and vendor unresponsive**
    - No schema, no API docs, vendor ghosting
    - **Response:** Too high risk, recommend switching systems
3. **Client wants real-time integration but system can't push data**
    - Behind firewall, no API, no database access
    - **Response:** Only batch integration possible, or client must upgrade system
4. **Data quality is terrible (missing 50%+ required fields)**
    - No buyer TINs, no product codes, inconsistent formats

    - **Response:** Client must fix data source first, we can't invent data

5. **Client wants "try before you buy" integration**
    - Wants full integration built before committing to paid plan
    - **Response:** Free plan = manual CSV upload only, integration requires paid plan commitment

---

Pricing Decision Tree

```
Start
  │
  ├─ Does pre-built adapter exist?
  │   ├─ Yes → RM 0 (installation support included)
  │   └─ No → Continue
  │
  ├─ Can client implement API themselves?
  │   ├─ Yes, they have dev team → RM 0 (API docs provided)
  │   └─ No → Continue
  │
  ├─ Integration complexity?
      ├─ Low (2-4 hours) → RM 800 - RM 1,500
      ├─ Medium (4-12 hours) → RM 1,500 - RM 4,500
      └─ High (12-40 hours) → RM 4,500 - RM 15,000

Additional Charges:
  ├─ On-site work → +RM 500/day + travel costs
  ├─ Urgent delivery (< 1 week) → +50% premium
  ├─ Ongoing maintenance → RM 500-2,000/month retainer
  └─ Training (beyond 1-hour handover) → RM 400/hour
```

Integration Proposal Email Template

```
Subject: E-Invoice Integration Proposal for [Client Name]

Hi [Client Name],

Thank you for providing detailed information about your current system.
I've completed the integration assessment and have good news — we can
connect your [System Name] to the e-invoice platform.
```

INTEGRATION SUMMARY

Current System: [System name & version]
Integration Method: [API / Database connector / File export / Other]
Complexity: [Low / Medium / High]
Timeline: [X] weeks from project kickoff

WHAT WE'LL DELIVER

✅ Custom adapter that connects [System] to e-invoice platform
✅ Automated data extraction ([Real-time / Hourly / Daily])
✅ Field mapping and validation logic
✅ Error handling with email notifications
✅ Testing with your sample invoice data
✅ Deployment to [production environment]
✅ 1-week post-deployment support
✅ Integration documentation

INVESTMENT

Development (fixed price): RM [Amount]

OR

Hourly billing: [X-Y] hours @ RM 400/hour

Payment Terms: 50% upfront, 50% upon successful deployment

TIMELINE

Week 1: Kickoff + requirements validation
Week 2-[X]: Development + testing
Week [X+1]: Deployment + go-live

Week [X+2]: Post-deployment support

Target Go-Live: [Date]

NEXT STEPS

If you'd like to proceed:

1. Review the attached Statement of Work (SOW)
2. Sign and return the SOW
3. We'll send the first invoice (50% deposit)
4. Schedule kickoff meeting

Questions? I'm happy to jump on a call to discuss further.

Best regards,
[Your name]

Document Control

Version: 1.0
Last Updated: May 20, 2026
Next Review: August 20, 2026
Owner: SteadyDevs Technical Team

SteadyDevs Solutions
SSM: 202603092285
Internal Use Only — Confidential